

Chat Context Export

Platform: ChatGPT

Exported: 13/09/2026, 15:22:31

Conversation ID: 6a9d49c6-0ae0-83e9-819e-977749e862bd

Conversation

User

Pasted text(20260906-154748).txtDocument

Assistant

Yes — and this failure actually exposes an important mistake in the tests I gave you. The tests themselves are correct as tests of the JA3 specification, but I was wrong to imply they should pass against your current implementation. Your output proves that your current implementation does not perform GREASE filtering. Salesforce's reference implementation explicitly filters GREASE values from: cipher suites, extension IDs, supported groups, EC point formats. GitHub+1 For example, Salesforce's includes: and its excludes values in that table. Its extension processing similarly excludes GREASE extension IDs. GitHub Your three failures are therefore meaningful Your test: is exactly showing that is currently serializing GREASE instead of ignoring it. Likewise: versus: and: are precisely the behavior the Salesforce implementation says should be filtered. So 94% passing is not a sign that the tests are bad. It means we've uncovered a missing JA3 requirement. But there's an important architectural question I would not immediately modify until we look at your parser. Salesforce filters GREASE while extracting the JA3 fields from the ClientHello. Your architecture may intend: If your parser is supposed to produce already-normalized , then GREASE filtering belongs in the parser. If is intended to represent the raw contents of ClientHello, then filtering belongs in . That's why I specifically want to inspect your before telling you to change anything. And there's another reason I'm being more careful now: the tests we added should test the behavior at the correct layer, not accidentally force an architectural change. What I am confident about TestConfidenceRFC 1321 MD5 vectors100%Salesforce official JA3 vectors100%JA3 field ordering/formatting100%GREASE should be ignored by JA3100%Your three GREASE tests correctly detect missing filtering100%GREASE must specifically be filtered inside Not yet The last point is the only thing we should not assume. Salesforce explicitly states that JA3 ignores GREASE so that clients using GREASE still produce the same fingerprint. GitHub So don't delete those three tests. They are doing exactly what we wanted: catching a real incompatibility. If you give me your current / (or upload them), I'll trace the exact data flow and tell you whether GREASE filtering should be added to the parser or , and then I'll give you the minimal correct patch.

Code Block 1 (text)

```
0x0a0a = 2570
0x1a1a = 6682
...
0xfafa = 64250
```

Code Block 2 (text)

```
without GREASE:
771,4865-4866-4867,,,

with GREASE:
771,2570-4865-6682-4866-4867,,,
```

Code Block 3 (text)

```
without GREASE:
771,,0-10-11,,
```

Code Block 4 (text)

```
with GREASE:
771,,2570-0-6682-10-11,,
```

Code Block 5 (text)

```
expected:
771,,,23-24,

actual:
771,,,2570-23-6682-24,
```

Code Block 6 (text)

```
raw ClientHello
☒
parser
☒
ClientHelloData
☒
compute_ja3()
```

User

I have added a file to git add it was new file how to remove it I will delete it but I think it may stay in the git history

Assistant

Yes, but there are two different things here: If you only did If the file is newly created and you have not committed yet, simply do: This removes it from the staging area but does not delete the file. Then delete the file: Or in one step: Then check: The file will be gone and it will not enter Git history at all, because you never committed it. If you already committed it That's different. If the file was committed: The file will disappear from the current version, but technically it will remain in Git history in the earlier commit. If you need to completely remove it from Git history, that's possible with (or BFG), but you generally shouldn't rewrite history unless the file contains something sensitive or you have another specific reason. In your situation You said: "I have added a file to git add it was new file" If you mean you literally only ran: and haven't committed: You're completely safe. Just run: It will not be in Git history.

Code Block 1 (text)

```
git restore --staged path/to/file
```

Code Block 2 (text)

```
rm path/to/file
```

Code Block 3 (text)

```
git restore --staged path/to/file  
rm path/to/file
```

Code Block 4 (text)

```
git status
```

Code Block 5 (text)

```
git rm path/to/file  
git commit -m "Remove accidentally added file"
```

Code Block 6 (text)

```
git add somefile
```

Code Block 7 (text)

```
git restore --staged somefile  
rm somefile
```

User

(base) ❯ code git:(main) ❯ git status On branch main Your branch is ahead of 'origin/main' by 2 commits.
(use "git push" to publish your local commits) Changes not staged for commit: (use "git add <file>..." to

update what will be committed) (use "git restore <file>..." to discard changes in working directory)
modified: cpp/CMakeLists.txt modified: cpp/tests/test_fingerprints.cpp Untracked files: (use "git add
<file>..." to include in what will be committed) cpp/tests/ja3_test.cpp no changes added to commit (use
"git add" and/or "git commit -a") I do not want the ja3_test.cpp

Assistant

Since is untracked, it has not been committed and is not in Git history. You can simply delete it: Then
verify: You should see only: Important Your current status says: So it is not staged and not committed.
Deleting it now means Git will have no record of it whatsoever. You do not need , because it isn't staged.

Code Block 1 (text)

```
rm cpp/tests/ja3_test.cpp
```

Code Block 2 (text)

```
git status
```

Code Block 3 (text)

```
modified: cpp/CMakeLists.txt  
modified: cpp/tests/test_fingerprints.cpp
```

Code Block 4 (text)

```
Untracked files:  
cpp/tests/ja3_test.cpp
```